

Contents

CerviQ Developer Documentation	2
1. Problem and project goals	2
2. Overall approach and architecture	2
3. Structural game model	3
Position, Direction, and Action	3
Tile and GameMap	3
Worm and WormStats	4
GameState	4
4. Core game engine	4
Local movement and growth	4
One simultaneous tick	4
Collision resolution and DeathReason	5
Food removal and statistics	5
5. Scenarios, maps, and validation	5
Static Scenario versus dynamic GameState	5
Built-in scenarios	6
Scenario validation	6
6. Food lifecycle	6
Generic food operations	6
Adaptive interactive food	6
Training and evaluation separation	7
7. Controllers, heuristic agents, and registry	7
Heuristic agents	7
AgentRegistry	8
8. Application and GUI architecture	8
Dynamic setup and capacity	8
Session construction	8
9. Watch Agents internals	9
Snapshot history and restart	9
Selection, HUD, and debug	9
10. Play vs Agents internals	9
11. Viewport and camera	10
12. Q-learning core and evolution	10
Generic QLearning.Core	10
V1 to V4	11
V1	11
V2	11
V3	11
13. Final V4 design	11
Compact RLState	11
Reusable search context	11
Multi-opponent threat approximation	12
Reusable action analysis	12
Loop detection and history	12
Tail connectivity	12
V4 reward	12
Final fallback policy	13
14. Training architecture	13
Configuration and generic loop	13
Diverse training environment	14
Final V4 executable	14
Configurable training executable	14
15. Evaluation architecture	15
Global metrics	15

Worm-specific metrics	15
16. Persistence and model files	15
17. Repository and source structure	16
18. Where is this implemented?	17
19. Extension points	17
Add a heuristic agent	17
Add a scenario or change capacity	18
Change interactive food configuration	18
Add a Q-learning version	18
20. Tests and useful commands	18
21. Known limitations and audit risks	19

CerviQ Developer Documentation

1. Problem and project goals

CerviQ models multiple continuously moving worms on a shared rectangular grid. Every living worm chooses one relative action per tick, all worms move simultaneously, and collision resolution determines the survivors. Food adds growth and a recurring navigation objective. The same rules support human play, AI visualization, tabular reinforcement-learning training, and repeated evaluation.

The implementation has four central goals:

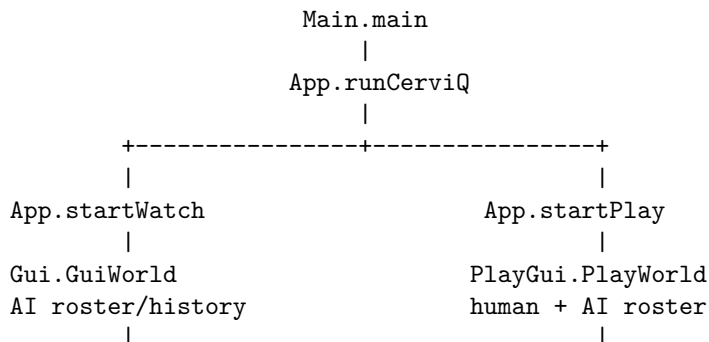
1. Keep the game transition rules independent of the graphical interface and individual controller implementations.
2. Make heuristic and learned agents interchangeable through one controller type.
3. Reuse one generic tabular Q-learning algorithm while evolving state and reward design from V1 through V4.
4. Support both the original two-worm experimental setup and larger interactive rosters without silently changing the supplied training distribution.

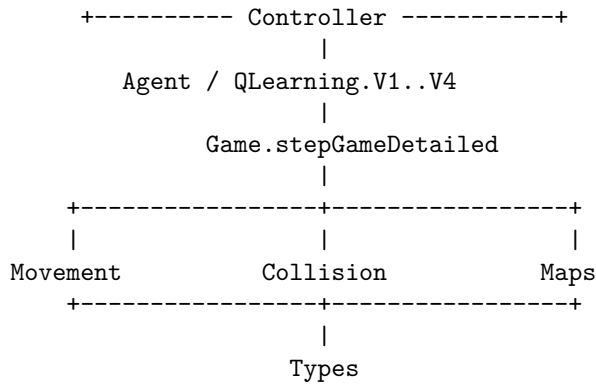
CerviQ is not a stop-and-start Snake implementation. Every living worm advances one cell on each simulated tick. Its only actions are **TurnLeft**, **GoStraight**, and **TurnRight**; there is no stop or 180-degree reverse action.

2. Overall approach and architecture

Types, **Maps**, **Movement**, and **Collision** provide the core data and pure operations. **Game.stepGameDetailed** combines them into the authoritative tick transition for a supplied list of actions. Random food placement and random agent choices remain in **IO**; the transition itself is deterministic once the state and actions are known.

The Cabal library exposes the modules under **src/**; the four files under **app/** are thin executable entry points. **Controller.collectActions** connects worm IDs to agents. **Gui** and **PlayGui** use that interface for interactive Watch and Play sessions. **Training** and **Evaluation** drive the same game engine without **Gloss**; **app/TrainModel.hs** and **app/EvaluateModels.hs** expose configurable command-line entry points. Version-specific Q-learning modules provide an encoder and reward through **QLearning.Core.QLearningSpec**; **QLearning.Core** owns the table and algorithm.





Scenarios -> Scenario -> GameState
 TrainingEnvironment -> Training -> QLearning.Core -> Game
 Evaluation -----> Game
 Viewport ----- rendering/camera only

TrainV4 / TrainModel -> TrainingEnvironment -> Training
 EvaluateModels -----> Evaluation

Randomness is intentionally outside `Game.stepGameDetailed`:

- random heuristic decisions use `System.Random.randomRIO` in `Agent`;
- food placement uses `randomRIO` in `Game`;
- environment sampling uses `randomRIO` in `TrainingEnvironment`;
- Gloss input/update/drawing is coordinated by `App`.

3. Structural game model

All fundamental types are defined in `Types`.

Position, Direction, and Action

`Position` is `(Int, Int)`. Coordinates are zero-based, X grows from left to right, and Y grows from top to bottom. Gloss uses Y increasing upward, so `Gui.mapPositionToScreen` and `Viewport.viewportPositionToScreen` invert the Y axis during rendering.

`Direction` is absolute: North, East, South, or West. `Action` is relative to a worm's current direction:

Action	Movement
TurnLeft	Rotate 90 degrees left, then advance one cell
GoStraight	Preserve direction, then advance one cell
TurnRight	Rotate 90 degrees right, then advance one cell

`Movement.applyAction` performs the rotation. `Movement.directionToAction` converts the Play UI's requested absolute direction into an `Action` and returns `Nothing` when the request would require a direct reverse.

Tile and GameMap

`Tile` has `Wall`, `Empty`, `Food`, and `Poison`. `GameMap` stores dimensions and `mapTiles :: Map Position Tile`. The map is sparse:

- `Maps.tileAt` returns `Nothing` outside the rectangle;
- an absent in-bounds map entry means `Empty`;
- `Maps.setTile` inserts an explicit tile;
- `Maps.removeTile` restores the implicit empty state.

`Maps.fromAsciiMap` parses #, ., F, and P. It rejects empty input, empty rows, unequal row widths, and unknown characters. Poison is fully represented, rendered, and collision-tested, but no built-in `Scenarios` map contains P.

Worm and WormStats

Worm stores:

- `wormId`, unique within a valid scenario;
- `wormBody`, ordered head first and tail last;
- `wormDirection`, the current absolute orientation;
- `wormAlive`;
- `wormStats`, containing `foodEaten`, `kills`, and `age`.

Every living worm must have a non-empty body because `Movement.wormHead` fails on `[]`. Scenario validation enforces this initially. Dead worms keep their final bodies and statistics in `gameWorms`, but movement, occupancy, safety, and collision calculations filter to living worms. A dead body is therefore not a persistent obstacle.

GameState

`GameState` contains the complete changing simulation:

Field	Meaning
<code>gameMap</code>	Current map, including changing food tiles
<code>gameWorms</code>	Every participant, living or dead
<code>gameTick</code>	Number of completed global transitions
<code>gameHeadHistory</code>	Recent head positions keyed by worm ID

`Game.initialHeadHistory` begins each list with the starting head. `Game.recordHeadHistory` prepends the current head and retains at most 256 positions. `Game.stepGameDetailed` passes living and dead worms to it, so after a death the final head continues to be prepended on later global ticks while another worm remains alive. V4 only examines history for the living controlled worm, so repeated dead-head entries do not influence that worm's decisions.

The type system does not enforce all game invariants. In particular, arbitrary callers can construct duplicate IDs, empty bodies, or overlapping segments. The intended construction path is through validated `Scenario` values.

4. Core game engine

Local movement and growth

`Movement.moveForward` changes a coordinate by one cell. `Movement.headAfterAction` predicts the destination for a relative action. `Movement.moveWormAfterAction` applies the turn and delegates body movement to `Movement.advanceBody`.

`advanceBody` always prepends the new head. For a normal move it removes the old tail, preserving length. For a growing move it retains the old tail, increasing length by one. Growth is decided from the pre-move map by `Game.wormWillEatFood`.

One simultaneous tick

`Game.stepGameDetailed` is the central transition:

1. Split `gameWorms` into living and already-dead worms.
2. For each living worm, use `Game.actionForWorm`; a missing assignment defaults to `GoStraight`.
3. Determine growth from the candidate head and current food.
4. Pass every `(grows, action, worm)` tuple to `Collision.simulateTurn`.
5. Remove food only under surviving moved heads.

6. Update survivor/death statistics and alive flags.
7. Recombine survivors, newly dead worms, and previously dead worms.
8. Record head history and increment `gameTick`.

`Collision.simulateTurn` first calls `Collision.futureWorms` for all prepared moves. It then evaluates every moved worm against that same future collection. No worm receives a sequential-update advantage.

The resulting `gameWorms` list is grouped as survivors, newly dead worms, then already-dead worms. Code should identify participants by `wormId`, not assume that list order remains stable after deaths.

`Game.stepGame` returns only `gameStepState`; `Game.stepGameDetailed` also returns (`wormId`, `DeathReason`) events. Evaluation uses the detailed form.

Collision resolution and `DeathReason`

`Collision.deathReason` uses this priority:

1. `HitWall` for a wall or out-of-bounds head;
2. `HitPoison` for a poison tile;
3. `HitOwnBody` for overlap with the worm's non-head body;
4. `HeadToHead [Int]` for two or more moved heads sharing a cell;
5. `HitOtherBody killerId` for overlap with another worm's non-head body.

Every worm whose moved head shares a head-to-head position dies. The associated ID list contains its own ID plus all other heads at that position. Head-to-head deaths do not credit a kill.

`Game.killerFromDeath` extracts an ID only from `HitOtherBody`. That ID is counted by `Game.applyKillStats`; a body owner can receive the kill even if it also dies during the same simultaneous tick.

Because tails are changed before collision checks, a non-growing worm's vacated tail is no longer occupied. A growing worm retains the tail, so it remains an obstacle in the future state.

Food removal and statistics

Food growth is prepared before movement, but `Game.removeEatenFood` receives only survivors. A worm that reaches food and dies on that tick neither removes the food nor gains `foodEaten`.

Every survivor and every newly dead worm gains one age tick. Survivors gain one food statistic when their moved head occupies food on the previous map. Kills are applied to survivors and newly dead worms. Already-dead worms no longer change.

5. Scenarios, maps, and validation

Static Scenario versus dynamic `GameState`

`Scenario.Scenario` is static configuration. It contains:

- `scenarioName` and `scenarioDescription`;
- `scenarioMap`;
- `scenarioWorms`, the original/default starts;
- `scenarioExtraWorms`, optional interactive starts;
- `scenarioBaseFoodCount` and `scenarioMaximumFoodCount`.

It deliberately contains no controllers or runtime timing. `GameState` is a copy that changes as worms move, food is removed/spawned, statistics accumulate, history grows, and ticks advance.

`Scenario.scenarioInitialState` uses only `scenarioWorms`. All built-in scenarios keep the original two worms in this field, preserving the historical training and evaluation environment.

`Scenario.scenarioInitialStateForWormCount` takes a prefix of `Scenario.scenarioAvailableWorms`, which concatenates defaults and extras. It is used by interactive modes to request the exact configured count.

Built-in scenarios

Binding	UI name	Size	Default + optional starts	Interactive food base/max
<code>Scenarios.arenaScenario</code>	Arena	20 x 10	2 + 4 = 6	2 / 4
<code>Scenarios.caveScenario</code>	Cave	30 x 15	2 + 6 = 8	2 / 5
<code>Scenarios.corridorScenario</code>	Corridors	30 x 15	2 + 4 = 6	2 / 4
<code>Scenarios.largeCaveScenario</code>	Large Cave	60 x 36	2 + 7 = 9	3 / 7

Arena, Cave, and Corridors are literal ASCII maps. Large Cave is deterministic: `Scenarios.largeCaveCell` derives each tile from fixed boundary, wall, and food coordinates, and the resulting rows are parsed by `Maps.fromAsciiMap`. It is not a procedural/random map generator.

Every built-in default start uses a length-three Worm 1 and Worm 2. Optional starts use IDs 3 upward. `Scenarios.allScenarios` is the menu registry.

Scenario validation

`Scenario.validateScenario` validates the complete default-plus-optional list, not only the currently selected prefix. `Scenario.validateWormConfiguration` requires:

- at least one worm;
- every worm initially alive;
- every body non-empty;
- unique IDs;
- no overlapping body positions;
- all segments inside the map;
- all segments on `Empty` tiles.

It also checks a non-negative food base and a maximum not below the base. `scenarioInitialState` and `scenarioInitialStateForWormCount` validate the complete scenario before producing a state. The four current built-ins pass with capacities 6, 8, 6, and 9 respectively.

Validation does not check segment adjacency, whether direction agrees with body orientation, zeroed starting statistics, or consecutive IDs. The current GUI labels roster positions as Worm 1 through Worm 9 and number-key selection uses actual IDs; custom interactive scenarios should therefore preserve ordered, consecutive IDs even though the validator only requires uniqueness.

6. Food lifecycle

CerviQ deliberately has separate generic and interactive food paths.

Generic food operations

`Game.freeFoodPositions` selects in-bounds `Empty` cells not occupied by any living worm body. `Game.randomFoodPosition` chooses uniformly from those cells. `Game.maintainFoodCount target` adds food until the current count is at least the target or no legal cell remains. It does not remove surplus food.

`Game.clearAllFood` removes all `Food` entries from the sparse map. `Game.randomizeFoodCount target` clears first and then calls the ordinary maintainer. Training uses this path with `Config.maxFoodCount = 1`.

Adaptive interactive food

`App.startWatch` and `App.startPlay` compute:

```
scenarioFoodCountForWorms n scenario
= min maximum (base + (n - 1) div 2)
```

where `n` is the configured starting worm count, including the human in Play.

Scenario	Targets by total starting worms
Arena	2:2, 3:3, 4:3, 5:4, 6:4
Cave	2:2, 3:3, 4:3, 5:4, 6:4, 7:5, 8:5
Corridors	2:2, 3:3, 4:3, 5:4, 6:4
Large Cave	2:3, 3:4, 4:4, 5:5, 6:5, 7:6, 8:6, 9:7

Both start functions call `Game.randomizeFoodCountAwayFromWorms`, which clears authored map food and creates the computed number of random items. The target is stored in `Gui.guiFoodTarget` or `PlayGui.playFoodTarget`; it does not decrease when worms die. `Gui.computeNextSnapshot` and `PlayGui.stepPlayWorld` call `Game.maintainFoodCountAwayFromWorms` after every tick.

`Game.randomFoodPositionAwayFromWorms` remains intentionally simple. It starts from ordinary legal free cells and prefers those with Manhattan distance at least `Game.interactiveFoodHeadDistance = 3` from every living head. If the preferred set is empty, it chooses from all legal free cells. There is no BFS fairness measure, farthest-point selection, optimization, or explicit spacing between food items.

Training and evaluation separation

`TrainingEnvironment.randomizeInitialFood` calls `Game.randomizeFoodCount` `Config.maxFoodCount`, so training clears authored food and starts each episode with one ordinary random food item. Subsequent `QLearning.Core.stepEnvironmentWithOpponents` calls `maintain` a minimum of one through the ordinary spawner. Interactive multi-worm changes do not alter this distribution.

`Evaluation.runEpisode` is different: it calls `Game.maintainFoodCount 1` on the supplied initial state without clearing it. If the caller passes an unmodified built-in `scenarioInitialState`, all authored food remains until consumed; one item is maintained only after the count drops below one. This is an existing evaluation-methodology distinction, not adaptive interactive food.

7. Controllers, heuristic agents, and registry

`Types.Agent` is:

```
type Agent = GameState -> Worm -> IO Action
```

The agent sees the complete state and the worm it controls. `IO` permits random selection. `Types.Controller` currently has only `AI Agent`.

`Controller.runController` looks up a living worm by ID and runs its agent. A dead or missing assignment produces `GoStraight`, allowing assignment lists to remain stable after deaths. `Controller.collectActions` applies this operation to every configured (`wormId`, `Controller`) pair.

Heuristic agents

Function	Behavior
<code>Agent.randomAgent</code>	Uniform random choice across all three actions
<code>Agent.safeRandomAgent</code>	Uniform random immediately safe action; random fallback if trapped
<code>Agent.greedyFoodAgent</code>	Minimize Manhattan distance to nearest food, ignoring safety
<code>Agent.safeGreedyFoodAgent</code>	Minimize food distance among immediate safe actions
<code>Agent.safeHunterAgent</code>	Minimize distance to the nearest living enemy head among safe actions

`Agent.enemyWorms` and `Agent.nearestEnemyHead` inspect all other living worms, not a fixed `Worm 2`. `Agent.isSafeAction` rejects walls, poison, self-overlap, and every position currently occupied by another living worm. It simulates the controlled worm as non-growing and does not move opponents. It can therefore be conservative about opponent tails and does not detect simultaneous head threats.

AgentRegistry

`AgentRegistry.AgentOption` combines a displayed name, controller, optional reward function, and optional `QLearning.Debug.AgentDebugProvider`. `AgentRegistry.loadAgentOptions` is the single GUI catalogue:

```
Random
Safe Random
Greedy Food
Safe Greedy Food
Safe Hunter
Q-learning V1
Q-learning V2
Q-learning V3 10k
Q-learning V3 20k
Q-learning V3 20k + fallback
Q-learning V3 Diverse 30k
Q-learning V3 Diverse 30k + fallback
Q-learning V4 Diverse 30k
Q-learning V4 Diverse 30k + fallback
Q-learning V4 Reformed 30k
Q-learning V4 Reformed 30k + fallback
```

Pure and `+ fallback` entries may share the same loaded Q-table but use different policy constructors. Model files are loaded once at application startup.

8. Application and GUI architecture

`Main.main` delegates to `App.runCerviQ`. That function loads the registry and starts `Gloss playIO` at 60 update calls per second. `App.AppWorld` separates:

- `AppMenu MenuWorld;`
- `AppWatch MenuWorld GuiWorld;`
- `AppPlay MenuWorld PlayWorld.`

The retained `MenuWorld` restores setup selections when `Esc` leaves a session. It stores one scenario index, a dynamic list of `Watch` agent-option indices, and a dynamic list of `Play` opponent-option indices.

Dynamic setup and capacity

The application-wide `Watch` limit is nine. `App.watchAgentCapacity` restricts it further with `Scenario.scenarioMaximumWormCount`. `Watch` requires at least two.

`Play` reserves one start for the human. `App.playOpponentCapacity` is the smaller of eight and scenario capacity minus one. `Play` requires at least one opponent.

`App.watchSetupFields` and `App.playSetupFields` derive navigation fields from the list lengths. Add operations append a `Safe Greedy Food` selection; remove operations remove only the last item. Changing scenario invokes `App.normalizeMenuForScenario`, trimming both stored rosters if their new capacity is lower.

`App.watchAgentColors` defines nine stable roster colors. `Watch` zips actual worm IDs, colors, and selected options into `GuiAgent` values. `Play` uses the first color for the human and the remaining colors for `PlayOpponent` values.

Session construction

`App.startWatch:`

1. validates the selected count against minimum and capacity;
2. calls `Scenario.scenarioInitialStateForWormCount`;
3. resolves every selected `AgentOption`;
4. computes and randomizes the adaptive food target;

5. builds one `GuiAgent` per actual worm ID;
6. calls `Gui.initialGuiWorldWithFoodTarget`.

`App.startPlay` follows the same state/food path, treats the first selected worm as human, maps every remaining ID to a separately selected `PlayOpponent`, and calls `PlayGui.initialPlayWorldWithFoodTarget`.

9. Watch Agents internals

`Gui.GuiAgent` contains the worm ID, display name, controller, color, optional reward function, and optional debug provider. `Gui.guiControllers` converts the entire `[GuiAgent]` list to controller assignments. No Watch simulation path is limited to the first two worms.

`Gui.GuiWorld` stores:

- `guiCurrent`, a `GuiSnapshot` of state, producing actions, and rewards;
- `guiHistory` and `guiFuture`;
- `guiInitialState` and fixed `guiFoodTarget`;
- the complete `guiAgents` roster;
- selected worm ID and HUD mode;
- pause, speed, and elapsed-time accumulator.

`Gui.computeNextSnapshot` obtains every configured controller action, calls the normal game step, replenishes to `guiFoodTarget`, and calculates transition rewards for agents that supply reward functions. `Gui.guiGameFinished` is true only when no worm is alive, so a sole survivor continues.

Snapshot history and restart

`Gui.stepGuiBackward` moves the current snapshot onto `guiFuture` and restores the newest history entry. `Gui.stepGuiForward` replays the first future snapshot when available; only an empty future triggers new controller decisions. `Gui.restartGui` restores `guiInitialState`, clears both history lists, and pauses. Because `guiInitialState` already contains randomized interactive food, restart is repeatable within one session rather than a new random setup.

Selection, HUD, and debug

Keys 1-9 change `guiSelectedWormId` only when a configured `GuiAgent` has that ID. The selection controls overview details, debug provider, and viewport focus. Debug remains single-worm even in a nine-worm match.

`Gui.watchRosterColumnCount` uses one overview column for up to four agents and two for larger rosters. `Gui.drawSelectedWormOverview` reports the selected worm. `Gui.drawWatchDebug` hides the roster to give version-specific diagnostics the full panel.

`QLearning.Debug.AgentDebugInfo` is version-independent: described state lines, three Q-values, and the raw best-Q action set. Heuristics have no provider. The V2-V4 modules construct additional diagnostic rows manually.

10. Play vs Agents internals

`PlayGui.PlayOpponent` holds one opponent's ID, display name, controller, and color. `PlayGui.playOpponents` is a list, not a single opponent field.

`PlayGui.stepPlayWorld`:

1. calls `Controller.collectActions` for every opponent;
2. prepends (`playHumanWormId`, `playPendingAction`);
3. calls `Game.stepGame` once with the complete action list;
4. replenishes to the stored interactive food target;
5. calls `PlayGui.resultForState` across all opponent IDs;
6. resets pending input to `GoStraight`.

Result semantics are:

Human alive	Any opponent alive	PlayResult
Yes	Yes	PlayRunning
Yes	No	PlayerWon
No	Yes	PlayerLost
No	No	PlayDraw

`PlayGui.queueHumanDirection` converts absolute arrow input with `Movement.directionToAction`; illegal reversal leaves the previous pending action unchanged. There is one slot, not an input queue.

`PlayGui.restartPlay` restores `playInitialState`, resets input/result/timing, and pauses. There is no Play history. The opponent roster uses one column up to four opponents and two columns above four. `PlayGui.drawPlayViewportIndicators` iterates over every living configured opponent.

11. Viewport and camera

`Viewport.useViewport` returns true when map width exceeds 30 or height exceeds 18. The camera therefore applies to the 60 x 36 Large Cave; current 20 x 10 and 30 x 15 maps render completely.

The important constants are:

Constant	Value
<code>viewportWidthCells</code>	30
<code>viewportHeightCells</code>	18
<code>viewportMaxPixelWidth</code>	800
<code>viewportMaxPixelHeight</code>	640
<code>viewportMaxCellSize</code>	40

`Viewport.viewportAround` clamps the camera rectangle to the map. In Play, `PlayGui.humanFocusPosition` uses the human head. In Watch, `Gui.selectedWormPosition` uses the selected worm head; dead worms retain their final body, so focus can remain on a dead selected worm. Missing/empty focus falls back to map center.

`Gui.drawGuiViewportIndicators` draws every living non-selected worm with its color and numeric ID. `PlayGui.drawPlayViewportIndicators` draws every living opponent. `Gui.drawNearestFoodIndicator` chooses geometrically nearest food by squared Euclidean distance and labels it F.

Viewport state never enters `GameState`. Agents, training, evaluation, and Q-learning always see the whole board.

12. Q-learning core and evolution

Generic `QLearning.Core`

`QLearning.Core.QLearningSpec` state provides:

```
qEncodeState    :: GameState -> Worm -> state
qRewardForStep  :: GameState -> Worm -> GameState -> Worm -> Double
```

`QTable` `state` is `Map (state, Action) Double`. `qValue` treats an unseen pair as zero. `hasQValue` distinguishes absent evidence from an explicitly stored zero, which is essential for fallback policies.

`chooseActionEpsilonGreedy` chooses a uniform random action with probability epsilon; otherwise `bestQAction` chooses randomly among maximum-Q ties. `qLearningAgent` is greedy at runtime with no epsilon exploration.

`updateQValue` implements:

```
Q(s,a) <- Q(s,a) + alpha * (r + gamma * max Q(s',a') - Q(s,a))
```

For a terminal transition, `rlNextRlState` is `Nothing` and the future term is zero. `stepEnvironmentWithOpponents` collects all assigned opponent actions, removes any assignment for the controlled ID, steps the normal game, maintains the default food threshold, computes reward, and encodes a next state only when the controlled worm survives.

V1 to V4

Version	Encoded state	
V1	Three immediate danger bits, absolute food axes, absolute heading, safe-action count, one global space category	M
V2	Three danger bits, space category after each action, worm-relative forward/side food direction	S
V3	One <code>ActionQuality</code> per action plus relative geometric food direction	A
V4	Three action qualities, BFS food-path direction, loop status	A

V1

`QLearning.V1.RLState` combines `Danger` for each action, horizontal and vertical absolute `FoodDirection`, `currentDirection`, `Mobility`, and one `SpaceLevel`. `QLearning.V1.reachableArea` flood-fills from the head while treating walls, poison, the remaining own body, and all living opponents as blocked. Fixed area thresholds are `<= 3` trapped, `<= 10` tight, otherwise open.

V2

`QLearning.V2.relativeFoodDeltas` rotates the food displacement into the worm's forward/side coordinate frame. `reachableAreaAfterAction` hypothetically moves the controlled worm, including growth when the destination contains food, and computes area separately for left/straight/right. Space thresholds scale with body length: `area <= length` is trapped, `area <= 2 * length` is tight, and larger is open. Opponents remain static.

V3

`QLearning.V3.possibleEnemyNextHeads` unions safe next destinations for every living opponent. `analyzeAction` classifies a candidate as `Fatal`, `Contested`, `DeadEnd`, `Threatened`, or a forced/non-forced area class. V3 reuses the current threat set for its future robustness check rather than recalculating opponents after the hypothetical controlled move.

`QLearning.V3.QLearningAgentWithFallback` removes fatal, contested, and dead-end actions when an acceptable alternative exists. It compares only explicitly learned candidates and otherwise ranks encoded `ActionQuality`. This introduced the first hybrid learned/heuristic deployment policy.

V1-V3 share reward weights: +50 per food, +10 per credited kill, -100 on death, +2/-2 for moving closer to/farther from Manhattan-nearest food, and -1 per tick. Distance shaping is suppressed when food is eaten.

13. Final V4 design

Compact RLState

`QLearning.V4.RLState` stores exactly:

- `qualityLeft`, `qualityStraight`, and `qualityRight`;
- `foodPathDirection` as `FoodPathLeft`, `FoodPathStraight`, `FoodPathRight`, or `FoodPathUnavailable`;
- `loopStatus` as `NotRepeating` or `RepeatingLoop`.

Raw areas, path distances, future move counts, tail connectivity, and visit counts remain outside the table key. They are used by fallback and diagnostics without expanding the tabular state space.

Reusable search context

The internal `MapSearchBase` stores dimensions, static walls/poison, and food. `makeMapSearchBase` derives it once from sparse `mapTiles`. `SearchContext` adds living body occupancy and removes the controlled head so it can be a search origin.

`reachablePositions` uses `Data.Sequence` BFS. It marks cells visited on enqueue, preventing duplicate queue growth, and accepts a temporary blocker set for threat-aware searches.

`foodPathInfoWithContext` initializes BFS only with destinations of the three legal relative actions, then carries the first position and distance through the queue. Reaching any food returns the corresponding first `Action` and shortest current path length. Walls, poison, and all current living bodies are static obstacles; the search does not simulate future tail movement.

Multi-opponent threat approximation

`QLearning.V4.possibleEnemyNextHeads` iterates over every living worm except the controlled worm, then unions destinations of each opponent's current `Agent.safeActions`. The runtime analysis is therefore multi-opponent-aware.

It remains an approximation. Every possible safe destination is treated as threatened even though an opponent selects only one, and there is no minimax, probabilistic policy model, or deeper adversarial tree.

Reusable action analysis

`makeActionAnalysisContext` computes current safe actions, current enemy threats, static map search data, and recent history once. `analyzeState` reuses that context for all three candidates and returns `StateAnalysis` containing the compressed `RLState` plus three rich `ActionAnalysis` values.

For each candidate, `analyzeActionWithContext`:

1. classifies an immediately unsafe move as `Fatal`;
2. classifies a destination in current opponent threats as `Contested`;
3. moves the controlled worm hypothetically, including food growth;
4. recalculates opponent threats in that hypothetical state;
5. counts raw and threat-robust next actions;
6. computes ordinary and threat-aware reachable components;
7. tests tail connectivity;
8. computes post-action food path distance;
9. counts recent use of the proposed edge and destination.

No raw continuation produces `DeadEnd`; no continuation outside recalculated threats produces `Threatened`. Otherwise threat-aware area is categorized relative to moved length. `Forced...` means exactly one robust continuation.

Loop detection and history

`recentHeadPositions` uses the newest 256 history entries. `recentMovementEdges` turns newest-first positions into directed edges. `loopStatusFromHistory` marks `RepeatingLoop` when the most recent directed edge appeared earlier in the window.

Candidate analysis also counts earlier occurrences of the proposed edge and destination. During a detected loop, `preferLoopBreaking` first keeps minimum edge visits and then minimum destination visits.

Tail connectivity

`tailReachableFrom` cannot enter the occupied tail cell. Instead, it checks whether the threat-aware component reaches at least one neighboring cell and whether the tail itself is not threatened. A length-one worm is treated as connected. This is a topological preference, not proof that a safe time-aware path will follow a moving tail.

V4 reward

`QLearning.V4.rewardForStep` retains +50 food, +10 kill, -100 death, and -1 per tick. Its +2/-2 progress term compares BFS path distances rather than Manhattan distance. Losing all reachability is -2; gaining reachability is +2. Food consumption suppresses distance shaping.

Final fallback policy

`QLearning.V4.qLearningAgentWithFallback` performs this candidate filtering:

1. `preferNonHardUnsafe` removes `Fatal`, `Contested`, and `DeadEnd` if possible.
2. `preferNonCritical` removes critical-space candidates if possible.
3. `preferTailConnected` keeps tail-connected candidates if any exist.
4. `preferLoopBreaking` narrows candidates during a detected loop.

Only then are Q-values considered. Among surviving candidates with explicit entries (`QLearning.Core.hasQValue`), maximum Q wins. If no survivor has a learned entry, `fallbackAction` compares a lexicographic score:

1. action quality;
2. robust future moves;
3. threat-aware area;
4. shorter reachable food path;
5. fewer recent edge uses;
6. fewer recent destination visits.

The filters and ranking are deterministic for unequal candidates, but exact ties use `Agent.randomChoice`. More importantly, this is not pure reinforcement learning: a high-Q action removed by a prior safety/topology filter cannot win. The plain `QLearning.V4.qLearningAgent` remains selectable for comparison and uses raw greedy Q-values across all actions.

`QLearning.V4.v4DebugProvider` exposes the compact state and rich analysis. Its **Q best** field is the raw table best set, not the post-filter fallback choice.

14. Training architecture

Configuration and generic loop

`Training.TrainingConfig` contains controlled worm ID, episode/tick limits, learning rate, discount, epsilon start/minimum/decay, and progress interval.

`Training.defaultTrainingConfig` is:

Field	Value
<code>trainingWormId</code>	1
<code>trainingEpisodes</code>	10,000
<code>maxEpisodeTicks</code>	500
<code>learningRate</code>	0.1
<code>discountFactor</code>	0.9
<code>epsilonStart</code>	1.0
<code>epsilonMinimum</code>	0.05
<code>epsilonDecay</code>	0.9995
<code>progressInterval</code>	200

`TrainingEpisodeSampler` is `Int -> IO TrainingEpisodeSetup`. A setup provides the initial state, all opponent controller assignments, and scenario/opponent/ start labels.

`Training.trainEpisodeWithSetup` repeats:

1. encode current state;
2. choose with epsilon-greedy exploration;
3. call `QLearning.Core.stepEnvironmentWithOpponents`;
4. update the chosen Q entry;
5. stop on controlled-worm death or `maxEpisodeTicks`.

The epsilon is constant within an episode. `Training.nextEpsilon` applies multiplicative decay between episodes and clamps it to the minimum. `trainEpisodesWithSampler` always starts with `emptyQTable`.

Training does not terminate merely because every opponent dies. The controlled worm can continue until its own death or the tick limit. The recorded training last-standing value means sole survivor in the final state only.

Diverse training environment

`TrainingEnvironment.mixedTrainingEpisode` samples independently:

Dimension	Distribution
Scenario	Arena 40%, Cave 30%, Corridors 30%
Opponent	Safe Greedy Food 40%, Safe Hunter 30%, Safe Random 20%, Random 10%
Starting side	Original 50%, first two bodies/directions swapped 50%
Initial food	Authored food cleared, then one ordinary random food item

The sampler calls `Scenario.scenarioInitialState`, so it uses only each scenario's two default worms. `TrainingEnvironment.swapFirstTwoWormStarts` exchanges bodies/directions while preserving IDs and resets head history.

Opponent assignment is written generically for every non-controlled ID, but the current sampled state contains only one such worm. Neither Large Cave nor any `scenarioExtraWorms` participate. The current final model training path is therefore one-versus-one even though the runtime encoder can inspect multiple opponents.

Final V4 executable

`app/TrainV4.hs` overrides the default config to:

Field	Final V4 value
Episodes	30,000
Maximum ticks	500
Alpha	0.1
Gamma	0.9
Initial/minimum epsilon	1.0 / 0.05
Epsilon decay	0.99985
Progress interval	200

The command is `stack run cerviq-train-v4`. It trains `QLearning.V4.v4Spec`, not the final fallback filters. The fallback affects deployment after training, not exploratory action selection or Q updates.

Progress blocks include reward, tick, death/survival, food, kills, length, last-standing, sampler labels, Q-table size, elapsed time, and throughput. When all episodes finish, `Main.main` writes:

```
data/models/qlearning_v4_reformed_diverse_30k.txt
data/models/qlearning_v4_reformed_diverse_30k_stats.txt
```

There is no checkpoint, resume, seed, or command-line configuration interface.

Configurable training executable

`app/TrainModel.hs` provides `stack run cerviq-train -- VERSION [OPTIONS]` for V1, V2, V3, or V4. It uses the same `TrainingEnvironment.mixedTrainingSampler` and generic training loop. Its defaults are 1,000 episodes, 500 ticks, alpha 0.1, gamma 0.9, epsilon 1.0 down to 0.05 with decay 0.9995, and progress every 100 episodes. Options can change those values and the model output path.

The default model path is `data/models/demo_VERSION.txt`. The runner also writes `_stats.txt`, `_stats.csv`, and `_summary.txt` files derived from the same base name. It starts from an empty table, does not checkpoint or resume, does not create parent directories, and has no random-seed option. This command changes experiment scale and hyperparameters, not the scenario/opponent/start sampling distribution described above.

15. Evaluation architecture

`Evaluation.runEpisode` accepts an explicit tick limit, controller assignment list, and initial state. It calls `Controller.collectActions`, `Game.stepGameDetailed`, and ordinary `Game.maintainFoodCount`. An episode runs while at least one worm is alive and `gameTick < maxTicks`; it stops when all worms die or the limit is reached.

`Evaluation.runEpisodes` repeats the same supplied initial value. Random agent decisions and later food replenishment still use the global random generator; there is no controlled seed. `Evaluation.evaluateAgent` is a convenience path with `Evaluation.defaultEvaluationMaxTicks = 500` and Safe Greedy Food assigned to every other worm ID. That assignment is generic for multi-worm states.

Global metrics

`EvaluationSummary` reports:

- `avgFoodEaten`: total food across all worms per episode;
- `avgEpisodeLength`: final global tick;
- `avgKills`: total credited kills across all worms;
- `survivalRate`: fraction ending with at least one living worm.

Because evaluation continues while any worm lives, global survival usually means that the tick limit was reached. It is not the selected agent's survival.

Worm-specific metrics

`WormEvaluationSummary` reports food, food per 100 lived ticks, age distribution, kills, final length, survival, last-standing, and death reasons for one ID. `foodPer100TicksByWorm` uses total worm age as denominator, not total match time.

`Evaluation.recordLastStandingWorm` records an ID whenever it becomes the only living worm. The event remains recorded if that worm later dies. This differs from `Training.trainingEpisodeLastStanding`, which examines only the final state.

Wall, poison, own-body, other-body, and head-to-head rates are fractions of all evaluated episodes, not fractions only among deaths. Death events come directly from `GameStepResult`.

Evaluation is measurement infrastructure, not unit testing. The lower-level functions can be called directly from Haskell code or a project REPL.

`app/EvaluateModels.hs` adds a pairwise CLI:

```
stack run cerviq-evaluate -- \  
  --agent "Q-learning V4 Reformed 30k + fallback" \  
  --opponent "Safe Greedy Food" \  
  --scenario Arena \  
  --episodes 100
```

The wrapper always constructs the selected scenario's two default worms, randomly keeps or swaps their starts, clears authored food, places exactly one ordinary random item, and then calls `Evaluation.runEpisode`. Its defaults are Arena, 100 episodes, and a 500-tick limit. Controllers may be registry names or direct `v1:`, `v2:`, `v3:`, `v3f:`, `v4:`, or `v4f:` model references. Optional text reports record the complete setup and summaries, but the output directory must already exist. The command has no seed option.

16. Persistence and model files

`QLearning.Core.saveQTable` serializes the complete `Map (state, Action) Double` with `show`. `loadQTable` reads the file and parses it with `readMaybe`. Persisted state types therefore need compatible `Read` and `Ord` instances.

`AgentRegistry.loadAgentOptions` loads these seven tables for the GUI and registered-name evaluation:

```

data/models/qlearning_v1.txt
data/models/qlearning_v2.txt
data/models/qlearning_v3.txt
data/models/qlearning_v3_20k.txt
data/models/qlearning_v3_diverse_30k.txt
data/models/qlearning_v4_diverse_30k.txt
data/models/qlearning_v4_reformed_diverse_30k.txt

```

The format has no schema header or version tag. Renaming constructors or changing a derived state representation can make an existing file unparseable. Files are loaded synchronously at startup through repository-relative paths.

Files ending in `_stats.txt` contain Show output for training episode lists. The configurable trainer can additionally write CSV and summary files. `cerviq-evaluate` can write text reports below the locally chosen output path. The repository's `.gitignore` excludes `outputs/`, so reports kept there are local benchmark artifacts rather than distributed repository files. They are not automated test reports.

17. Repository and source structure

<code>cerviq.cabal</code>	exposed library, four executables, and test suite
<code>stack.yaml</code>	pinned Stackage snapshot
<code>Setup.hs</code>	Cabal Simple-build compatibility entry point
<code>app/</code>	
<code>Main.hs</code>	GUI executable entry point
<code>TrainV4.hs</code>	final 30,000-episode V4 training executable
<code>TrainModel.hs</code>	configurable V1-V4 training and output reports
<code>EvaluateModels.hs</code>	pairwise two-worm evaluation command
<code>src/</code>	
<code>App.hs</code>	menus, dynamic rosters, mode construction, playIO
<code>Types.hs</code>	positions, map, worm, state, controller types
<code>Maps.hs</code>	sparse-map queries/mutation and ASCII parsing
<code>Movement.hs</code>	directions, relative actions, body movement/growth
<code>Collision.hs</code>	simultaneous collisions and DeathReason
<code>Game.hs</code>	complete tick, food, statistics, head history
<code>Config.hs</code>	legacy/default food and timing constants
<code>Scenario.hs</code>	scenario data, capacity, validation, state creation
<code>Scenarios.hs</code>	four built-in deterministic environments
<code>Controller.hs</code>	controller lookup and action collection
<code>Agent.hs</code>	five heuristic agents
<code>AgentRegistry.hs</code>	GUI catalogue and model loading
<code>Gui.hs</code>	Watch simulation, history, HUD, shared rendering
<code>PlayGui.hs</code>	human plus N agents, results, Play rendering
<code>Viewport.hs</code>	camera and off-screen indicator geometry
<code>Training.hs</code>	generic trainer and progress statistics
<code>TrainingEnvironment.hs</code>	weighted two-worm episode sampler
<code>Evaluation.hs</code>	repeated episodes and evaluation metrics
<code>QLearning/</code>	
<code>Core.hs</code>	generic table, update, RL step, persistence
<code>Debug.hs</code>	version-independent diagnostic interface
<code>V1.hs</code>	absolute-food baseline
<code>V2.hs</code>	relative food and per-action space
<code>V3.hs</code>	action quality and first fallback policy
<code>V4.hs</code>	path/history/topology analysis and final policy
<code>data/models/</code>	persisted tables and ignored local training stats
<code>test/Spec.hs</code>	Tasty/HUnit deterministic unit-test suite
<code>outputs/evaluation/</code>	ignored local pairwise report destination
<code>docs/</code>	user, developer, and experiment documentation

`Config.playerWormId` and `Config.tickDelay` are not used by the current Gloss application loops. Runtime IDs are taken from scenarios and timing is driven by Gloss elapsed time. `Config.maxFoodCount` remains active in training, Q-learning environment steps, compatibility constructors, and evaluation replenishment, but not as the adaptive interactive target.

18. Where is this implemented?

Responsibility	Module and key functions/types
Domain model	<code>Types.Position</code> , <code>Direction</code> , <code>Action</code> , <code>Tile</code> , <code>GameMap</code> , <code>Worm</code> , <code>GameState</code>
Relative/absolute movement	<code>Movement.applyAction</code> , <code>Movement.directionToAction</code> , <code>Movement.headAfterAction</code>
Body movement and growth	<code>Movement.advanceBody</code> , <code>Movement.moveWormAfterAction</code>
Sparse maps	<code>Maps.tileAt</code> , <code>Maps.setTile</code> , <code>Maps.fromAsciiMap</code>
Simultaneous collision	<code>Collision.futureWorms</code> , <code>Collision.deathReason</code> , <code>Collision.simulateTurn</code>
Complete game tick	<code>Game.stepGameDetailed</code> , <code>Game.stepGame</code>
Food removal/free cells	<code>Game.removeEatenFood</code> , <code>Game.freeFoodPositions</code> , <code>Game.clearAllFood</code>
Ordinary random food	<code>Game.randomFoodPosition</code> , <code>Game.maintainFoodCount</code> , <code>Game.randomizeFoodCount</code>
Interactive food safeguard	<code>Game.randomFoodPositionAwayFromWorms</code> , <code>Game.maintainFoodCountAwayFromWorms</code>
Statistics and kills	<code>Game.updateSurvivorStats</code> , <code>Game.updateDeadWorm</code> , <code>Game.killerFromDeath</code>
Head history	<code>Game.initialHeadHistory</code> , <code>Game.recordHeadHistory</code> , <code>Game.resetHeadHistory</code>
Scenario capacity/food formula	<code>Scenario.scenarioMaximumWormCount</code> , <code>Scenario.scenarioFoodCountForWorms</code>
Scenario validation/state	<code>Scenario.validateScenario</code> , <code>Scenario.scenarioInitialState</code> , <code>Scenario.scenarioIni</code>
Built-in maps and starts	<code>Scenarios.allScenarios</code> , <code>arenaScenario</code> , <code>caveScenario</code> , <code>corridorScenario</code> , <code>largeCave</code>
Controller execution	<code>Controller.runController</code> , <code>Controller.collectActions</code>
Heuristic decisions	<code>Agent.safeActions</code> , <code>Agent.safeGreedyFoodAgent</code> , <code>Agent.safeHunterAgent</code>
Selectable agent catalogue	<code>AgentRegistry.loadAgentOptions</code> , <code>AgentRegistry.makeGuiAgent</code>
Top-level application	<code>Main.main</code> , <code>App.runCerviQ</code> , <code>App.handleAppEvent</code>
Dynamic roster limits	<code>App.watchAgentCapacity</code> , <code>App.playOpponentCapacity</code> , <code>App.normalizeMenuForScenario</code>
Session construction	<code>App.startWatch</code> , <code>App.startPlay</code> , <code>App.makePlayOpponent</code>
Watch stepping/history	<code>Gui.computeNextSnapshot</code> , <code>Gui.stepGuiBackward</code> , <code>Gui.stepGuiForward</code> , <code>Gui.restartGui</code>
Watch HUD/debug	<code>Gui.drawWatchOverview</code> , <code>Gui.drawWatchDebug</code> , <code>Gui.drawAgentDebug</code>
Human and N AI tick	<code>PlayGui.queueHumanDirection</code> , <code>PlayGui.stepPlayWorld</code>
Play result semantics	<code>PlayGui.resultForState</code> , <code>PlayGui.restartPlay</code>
Viewport/camera	<code>Viewport.viewportAround</code> , <code>Viewport.offscreenIndicatorPlacement</code>
Off-screen rendering	<code>Gui.drawGuiViewportIndicators</code> , <code>PlayGui.drawPlayViewportIndicators</code>
Generic Q-learning	<code>QLearning.Core.QLearningSpec</code> , <code>chooseActionEpsilonGreedy</code> , <code>updateQValue</code>
Q persistence	<code>QLearning.Core.saveQTable</code> , <code>QLearning.Core.loadQTable</code>
V1/V2/V3 encoding	<code>QLearning.V1.encodeState</code> , <code>QLearning.V2.encodeState</code> , <code>QLearning.V3.encodeState</code>
V4 path and state	<code>QLearning.V4.foodPathInfo</code> , <code>QLearning.V4.encodeState</code> , <code>internal.analyzeState</code>
V4 threat/topology policy	<code>internal.possibleEnemyNextHeads</code> , <code>tailReachableFrom</code> ; <code>QLearning.V4.qLearningAgentW</code>
Debug abstraction	<code>QLearning.Debug.AgentDebugProvider</code> , <code>QLearning.Debug.makeQDebugProvider</code>
Generic training	<code>Training.trainEpisodeWithSetup</code> , <code>Training.trainEpisodesWithSampler</code>
Training sampling	<code>TrainingEnvironment.mixedTrainingEpisode</code> , <code>mixedTrainingSampler</code>
Final training command	<code>app/TrainV4.hs</code> : <code>v4TrainingConfig</code> , <code>main</code>
Configurable training command	<code>app/TrainModel.hs</code> : <code>TrainOptions</code> , <code>runTraining</code> , <code>main</code>
Evaluation simulation	<code>Evaluation.runEpisode</code> , <code>Evaluation.runEpisodes</code> , <code>Evaluation.evaluateAgent</code>
Evaluation summaries	<code>Evaluation.summarizeResults</code> , <code>Evaluation.summarizeWormResults</code>
Pairwise evaluation command	<code>app/EvaluateModels.hs</code> : <code>EvaluationOptions</code> , <code>runEvaluation</code> , <code>main</code>
Automated unit tests	<code>test/Spec.hs</code> : <code>main</code> and the nine <code>TestTree</code> groups

19. Extension points

Add a heuristic agent

Implement `Agent` behavior, normally reusing `Agent.safeActions` or movement helpers. Register it with `AgentRegistry.heuristicAgentOption` inside `loadAgentOptions`. It then appears independently in every

Watch and Play roster row. Use `debuggableAgentOption` only when meaningful reward and state-debug providers exist.

Add a scenario or change capacity

Construct a `GameMap`, create starts with `Scenario.freshWorm`, and fill both `scenarioWorms` and `scenarioExtraWorms`. Add the value to `Scenarios.allScenarios`. Capacity is derived automatically from both lists.

Keep the default two-worm list unchanged if the scenario should remain compatible with current training methodology. Add extra starts only for larger interactive rosters. Use ordered IDs 1..N for current menu labels and number-key focus, validate all starts, and choose food base/maximum values explicitly.

Change interactive food configuration

Change `scenarioBaseFoodCount` and `scenarioMaximumFoodCount` for a scenario; `Scenario.scenarioFoodCountForWorms` applies the common scaling formula. Change `Game.interactiveFoodHeadDistance` or the `...AwayFromWorms` functions only when altering interactive placement policy. Do not change `Config.maxFoodCount` if the intention is only to tune interactive multi-worm sessions, because that constant belongs to training/evaluation compatibility paths.

Add a Q-learning version

Define an ordered/readable state, an encoder, and reward, then bind them through `QLearning.Core.QLearningSpec`. Reuse core training/persistence. Provide an `AgentDebugProvider`, add the module to `cerviQ.cabal`, train a compatible table, and register the file/controller in `AgentRegistry`. To use the generic CLI, also add version parsing and dispatch in `app/TrainModel.hs`; a fixed recipe can use a separate entry point and executable stanza similar to `app/TrainV4.hs`.

20. Tests and useful commands

The `cerviQ-tests` Cabal target runs `test/Spec.hs` with Tasty and HUnit. Its 59 test cases are organized into nine groups:

Group	Verified behavior
Movement	Relative turns, reverse rejection, head prediction, and body growth
Collision	Walls, poison, bodies, two-/three-way head collisions, and unaffected worms
Game	Missing actions, four-worm stepping, food growth, tail vacancy, kills, and statistics
Maps	ASCII parsing, dimensions, sparse empty cells, food, and poison
Scenario	Built-in validation, default states, capacities, maximum rosters, IDs, and invalid counts
Adaptive Food	Scaling, cap, clearing, head distance, dead-worm exclusion, random placement, and fallback
Play	Complete multi-opponent running/win/loss/draw result matrix
Q-learning Core	Unseen/known entries, tied best actions, and terminal/non-terminal Q updates
Evaluation	Sole-survivor detection and persistent last-standing recording

The suite exercises deterministic rules and invariants; the randomized food cases assert properties rather than fixed coordinates. It does not currently exercise Gloss rendering/input, Watch history, viewport geometry, controller IO, model file round trips, complete V1-V4 encoders/policies, or long stochastic training/evaluation behavior. Training and evaluation remain simulation and measurement infrastructure, not substitutes for unit tests.

Command	Purpose
<code>stack build</code>	Compile the library and all four executables with configured warnings
<code>stack test</code>	Build and run the 59 Tasty/HUnit unit tests
<code>stack run cerviQ</code>	Start the Gloss application
<code>stack run cerviQ-train-v4</code>	Start a full new training run and eventually overwrite its output paths

Command	Purpose
<code>stack run cerviq-train -- --help</code>	Show configurable V1-V4 training options
<code>stack run cerviq-evaluate -- --help</code>	Show pairwise evaluation options
<code>git diff --check</code>	Check edited text for whitespace errors

21. Known limitations and audit risks

- The supplied Q-learning models use tabular state abstraction. Distinct full positions can alias to one key, and unseen state-action pairs lack learned evidence.
- The final V4 runtime supports multiple opponents, but the repository training sampler and final training executable use the original two-worm scenarios. The provided Reformed model was not trained specifically for 6-9 worm games.
- V3/V4 fallback controllers combine learned values with hand-designed filters. They are hybrid policies, not pure learned policies.
- V4 opponent modeling is a conservative one-step destination union, not minimax or full adversarial planning.
- V4 BFS treats bodies as static for each analysis. Tail connectivity is a local topological heuristic rather than future simulation.
- Heuristic safety does not predict simultaneous opponent moves.
- Interactive food, training environments, evaluation replenishment, random agents, and tie-breaking have no exposed seed control.
- Lower-level `Evaluation.runEpisode` does not clear authored initial food, while training and `cerviq-evaluate` do. Direct callers must document the initial state they supplied.
- Restart restores the session's stored randomized initial state; it does not generate new food until a new session is started.
- Maps and starts are deterministic. Large Cave uses coordinate rules but is not procedural generation.
- Poison is engine support only in the current built-in scenario set.
- Scenario validation does not enforce body adjacency/orientation or the GUI's consecutive-ID convention.
- `Show/Read` model persistence has no schema/version metadata.
- The fixed final training recipe and configurable trainer both lack checkpoint/resume support and write results only after completion.
- Pairwise evaluation is restricted to the original two-worm scenario starts; it has report output but no seed control.
- Quantitative tables in `docs/experiments.md` were derived from local reports under ignored `outputs/`; the raw reports are not distributed by Git.
- Watch continues after one worm becomes sole survivor; it stops only when all worms are dead.